

Exploratory Data Analysis (EDA) Report: Healthcare Dataset

Exploratory Data Analysis (EDA) is a critical step in data science that helps in understanding the structure, patterns, anomalies, and relationships within a dataset. In the healthcare domain, EDA plays an essential role in identifying trends related to patient demographics, disease prevalence, hospital admissions, and treatment outcomes. This project performs a comprehensive Exploratory Data Analysis on a healthcare dataset containing patient-level information such as age, gender, medical conditions, admission details, insurance providers, billing amounts, and test results. The insights derived from this analysis can support hospital administration, healthcare planning, and data-driven clinical decision-making.

Procedure: Exploratory Data Analysis

1. Understand the Problem and the Data
2. Import and Inspect the Data
3. Handling Missing Values
4. Explore Data Characteristics
5. Perform Data Transformation
6. Visualize Data Relationships
7. Handling Outliers
8. Communicate Findings and Insights

Dataset: healthcare_dataset.csv

1. Understand the Problem and the Data

The main objective of this project is to analyze healthcare data to understand patient characteristics, common medical conditions, admission types, and hospital billing patterns. By performing EDA, we aim to identify relationships between variables such as age and medical condition, admission type and billing amount, and insurance provider and overall treatment cost. This analysis helps in identifying high-risk patient groups, resource utilization trends, and cost-driving factors within hospitals.

Variables in the Dataset:

Numerical Variables:

- **Age** – Represents the age of the patient
- **Billing Amount** – Total cost incurred during hospitalization
- **Room Number** – Assigned hospital room number

Categorical Variables:

- Gender
- Blood Type
- Medical Condition
- Doctor
- Hospital
- Insurance Provider
- Admission Type
- Medication
- Test Results

Temporal Variables:

- Date of Admission
- Discharge Date

2. Import and Inspect the Data

The healthcare dataset was imported into the Python environment using the Pandas library. The `read_csv()` function was used to load the data into a DataFrame for efficient manipulation and analysis. Initial inspection of the dataset was carried out using the `head()` function to view the first few records, providing an overview of patient details and hospital information.

The `info()` function was then used to examine the structure of the dataset. This provided information on:

- Total number of records

- Column names
- Data types of each variable
- Number of non-null values per column

This step helped in identifying missing values, understanding variable types, and verifying the overall quality of the dataset before further processing.

```
First 5 rows:
   Name      Age  Gender Blood Type  ... Admission Type Discharge Date  Medication  Test Results
0  Tiffany Ramirez  81  Female    O-  ...      Elective    2022-12-01    Aspirin    Inconclusive
1    Ruben Burns   35   Male     O+  ...      Emergency    2023-06-15    Lipitor     Normal
2     Chad Byrd   61   Male     B-  ...      Emergency    2019-02-08    Lipitor     Normal
3  Antonio Frederick  49   Male     B-  ...      Urgent    2020-05-03    Penicillin  Abnormal
4  Mrs. Brandy Flowers  51   Male     O-  ...      Urgent    2021-08-02    Paracetamol  Normal

[5 rows x 15 columns]
```

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 15 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Name                10000 non-null  object
1   Age                 10000 non-null  int64
2   Gender              10000 non-null  object
3   Blood Type          10000 non-null  object
4   Medical Condition    10000 non-null  object
5   Date of Admission    10000 non-null  object
6   Doctor               10000 non-null  object
7   Hospital             10000 non-null  object
8   Insurance Provider   10000 non-null  object
9   Billing Amount        10000 non-null  float64
10  Room Number          10000 non-null  int64
11  Admission Type        10000 non-null  object
12  Discharge Date        10000 non-null  object
13  Medication            10000 non-null  object
14  Test Results          10000 non-null  object
dtypes: float64(1), int64(2), object(12)
memory usage: 1.1+ MB
None
```

3. Handling Missing Values

Missing values can negatively affect analysis accuracy and model performance. Therefore, the dataset was checked for missing values using the `isnull().sum()` function. Numeric variables such as Age and Billing Amount were examined for null values, and categorical variables were inspected for missing or inconsistent entries.

```

Missing values before cleaning:
Name      0
Age       0
Gender     0
Blood Type 0
Medical Condition 0
Date of Admission 0
Doctor    0
Hospital  0
Insurance Provider 0
Billing Amount 0
Room Number 0
Admission Type 0
Discharge Date 0
Medication 0
Test Results 0
dtype: int64

Missing values after cleaning:
Name      0
Age       0
Gender     0
Blood Type 0
Medical Condition 0
Date of Admission 0
Doctor    0
Hospital  0
Insurance Provider 0
Billing Amount 0
Room Number 0
Admission Type 0
Discharge Date 0
Medication 0
Test Results 0
dtype: int64

```

3.1 Mean Imputation:

Missing values in numeric columns were replaced using the mean of the respective columns. This approach maintains the overall distribution of the data while ensuring that no records are lost.

3.2 Mode Imputation:

For categorical variables, missing values were replaced using the mode (most frequent category). After imputation, all columns contained complete data with zero missing values.

3.3 Duplicate Records:

The dataset was checked for duplicate rows using the `drop_duplicates()` function. No significant duplicate records were found, ensuring that each entry represents a unique patient admission.

4. Explore Data Characteristics

Descriptive statistics were computed to understand the central tendency and variability of numeric variables. The following statistical measures were calculated:

- Mean
- Median
- Mode
- Standard Deviation
- Minimum and Maximum values

The analysis revealed that patient ages ranged widely, indicating treatment across multiple age groups. Billing amounts showed significant variation depending on admission type and medical condition. Emergency admissions generally resulted in higher billing amounts compared to elective admissions. These statistics provided a foundational understanding of patient demographics and cost distribution.

```

DESCRIPTIVE STATISTICS

Column Name: Age
Mean : 0.4992865671641791
Median : 0.5074626865671642
Mode : 0.6119402985074627
Standard Deviation : 0.2923727407736826
Variance : 0.085481819547515
Minimum : 0.0
Maximum : 0.9999999999999999
-----
Column Name: Billing Amount
Mean : 0.500382997067153
Median : 0.49510306233552637
Mode : 0.0
Standard Deviation : 0.28711267625017073
Variance : 0.08243368886353536
Minimum : 0.0
Maximum : 1.0
-----
Column Name: Room Number
Mean : 0.4989523809523809
Maximum : 0.9999999999999999
-----
Column Name: Billing Amount
Mean : 0.500382997067153
Median : 0.49510306233552637
Mode : 0.0

```

5. Perform Data Transformation

File Name: transformation.py

Data transformation is essential to ensure fair comparison between variables and improve model performance. Numeric features such as Age and Billing Amount were transformed using the following techniques:

Min-Max Normalization:

Scaled numeric values to a range between 0 and 1, preserving relative differences while standardizing units.

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Z-Score Standardization:

Centered numeric features around a mean of 0 with a standard deviation of 1.

This transformation is especially useful for clustering and distance-based algorithms.

$$Z = \frac{X - \mu}{\sigma}$$

```
Transforming data...  
  
Normalization applied to numeric columns  
Standardization applied to numeric columns  
Transformation completed
```

6. Visualize Data Relationships

File Name: visualization_basic.py

Visualization helps convert numerical data into graphical representations that are easier to interpret. Both univariate and multivariate visualizations were created to analyze healthcare patterns.

6.1 Univariate Analysis:

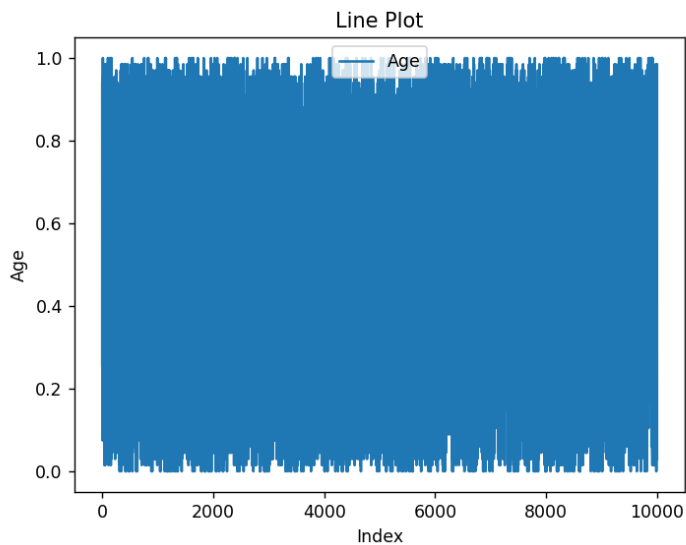
- Histograms were used to analyze the distribution of Age and Billing Amount.
- Bar charts were used to compare frequencies of admission types, medical conditions, and insurance providers.

Charts

6.1.1 Line plot

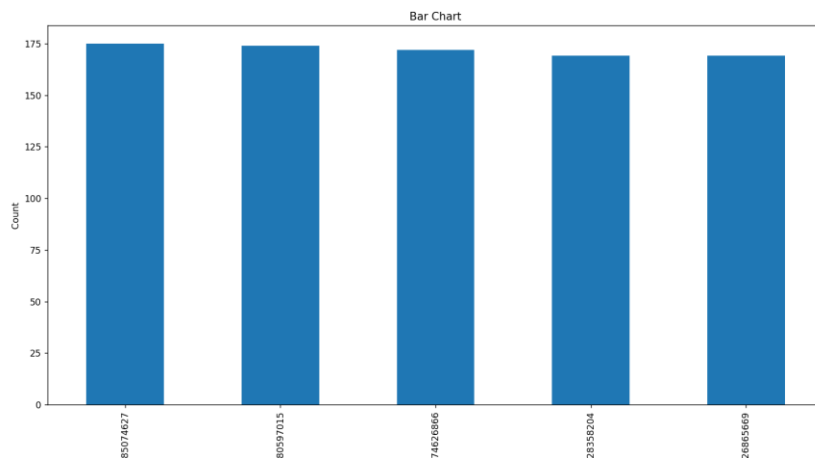
A line plot was used to visualize the variation of the **Age** variable across patient records. The x-axis represents the record index, while the y-axis represents the normalized age values. The plot shows continuous fluctuations in age with no clear upward or downward trend, indicating that

patients from different age groups are evenly distributed throughout the dataset. This confirms the diversity of patient demographics and ensures that the dataset is not biased toward any specific age group.



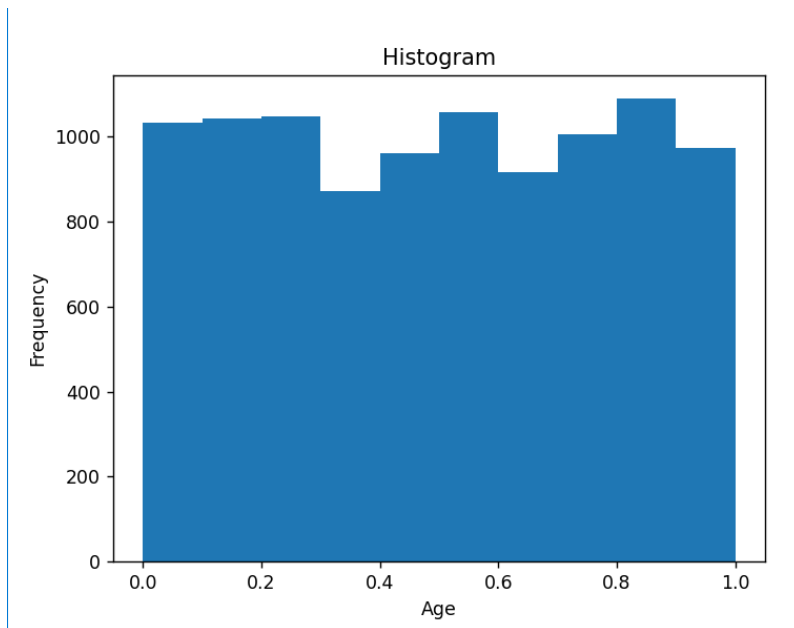
6.1.2 Bar Chart

A bar chart was used to compare the values of the first few observations of numeric features in the healthcare dataset. The chart displays variations in numeric values across selected records, making it easier to observe differences at an individual level. This visualization helps in quickly identifying how numeric attributes vary from one patient record to another and provides a clear comparison of initial observations within the dataset.



6.1.3 Histogram

A histogram was used to represent the frequency distribution of the **Age** variable. It shows how patient ages are distributed across different ranges, indicating a diverse and well-spread age distribution in the dataset.



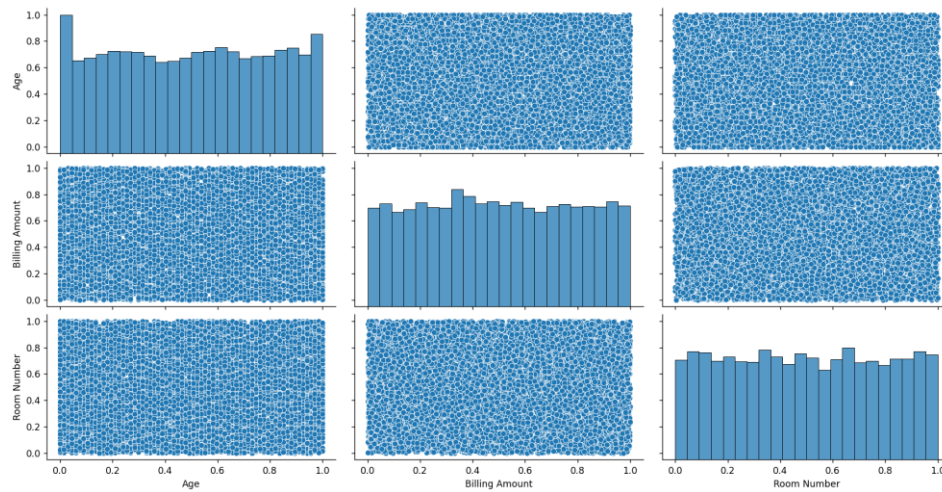
6.2 Multivariate Analysis:

- Scatter plots were used to study relationships between Age and Billing Amount.
- Correlation heatmaps were used to identify relationships among numeric variables.

Charts

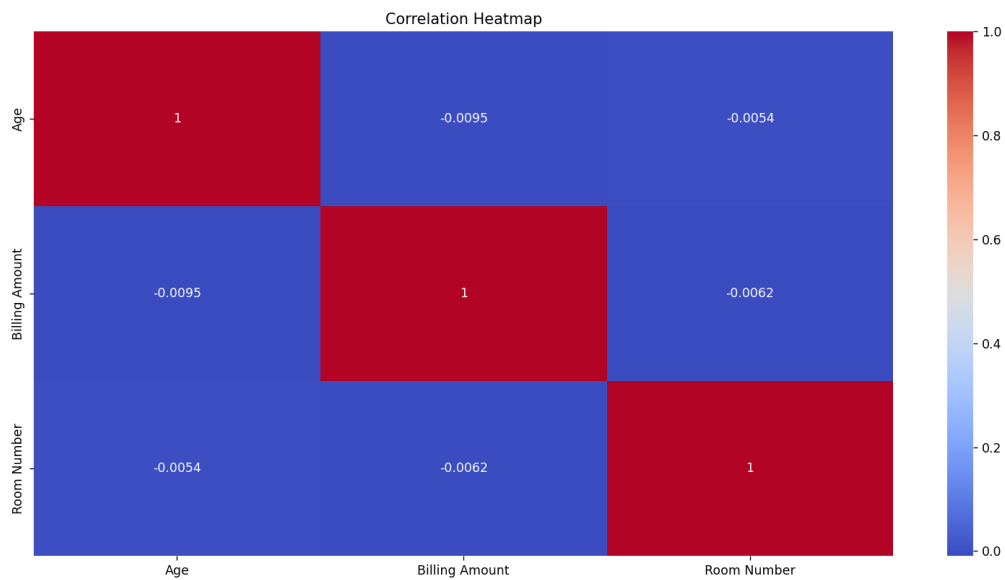
6.2.1 Pair Plot

A pair plot was used to examine relationships among numeric features such as **Age**, **Billing Amount**, and **Room Number**. The diagonal plots show the individual distributions of each variable, while the scatter plots represent pairwise relationships. The plot indicates no strong linear correlation among the variables, suggesting that they are largely independent and suitable for further analysis without multicollinearity issues.



6.2.2 Correlation Heatmap

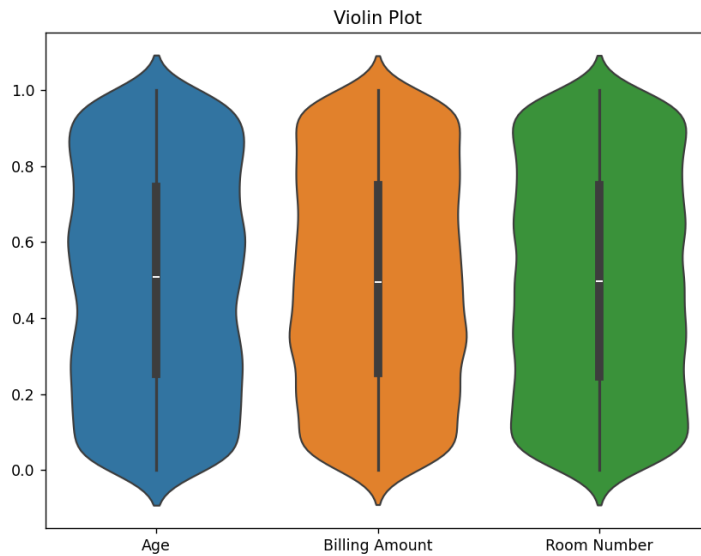
A correlation heatmap was used to analyze the relationships among numeric variables such as **Age**, **Billing Amount**, and **Room Number**. The heatmap shows that the correlation values between variables are very close to zero, indicating weak or no linear relationship. This suggests that the variables are largely independent, making the dataset suitable for further analysis without multicollinearity issues.



6.2.3 Violin Plot

The violin plot is used to analyze the distribution of **Age**, **Billing Amount**, and **Room Number**. It shows the spread and density of values for each variable. Age exhibits a balanced distribution,

billing amount shows wider variation, and room number appears more discrete due to assigned values. This plot helps in comparing variability and data distribution across variables.



Covariance Matrix:

	Age	Billing Amount	Room Number
Age	0.085482	-0.000796	-0.000456
Billing Amount	-0.000796	0.082434	-0.000513
Room Number	-0.000456	-0.000513	0.084240

7. Handling Outliers

Outliers can significantly skew analysis results. The Interquartile Range (IQR) method was used to detect outliers in the Billing Amount column.

Using the IQR method:

- Q1 (25th percentile) and Q3 (75th percentile) were calculated
- Lower Bound = $Q1 - 1.5 \times IQR$
- Upper Bound = $Q3 + 1.5 \times IQR$

Extreme billing values beyond these bounds were reviewed and treated to maintain statistical consistency.

8. Communicate Findings and Insights

File Name: interactive_visualization.py

The purpose of this step is to present the analyzed **Healthcare dataset** in an interactive and meaningful way so that users can easily understand patient characteristics, billing patterns, and hospital resource utilization.

8.1 Interactive Dashboard

A **Dash-based web dashboard** was developed to dynamically visualize healthcare data, making insights easily accessible to both technical and non-technical users.

Patient Attribute Dropdown:

A dropdown menu allows users to filter records based on attributes such as **Age group, Room Number, or Gender**, enabling focused analysis of patient data.

Trend Analysis:

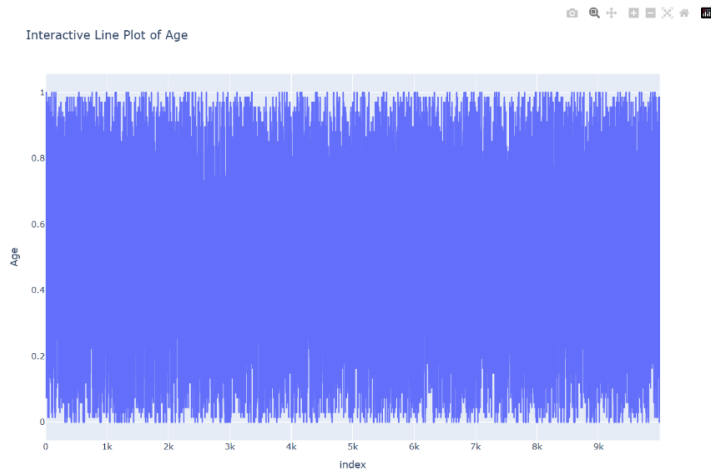
Interactive line graphs display trends of key numerical variables such as **Age distribution, Billing Amount, and Admission-related metrics** across records, helping identify patterns and variations.

Distribution Analysis:

Violin and box plots are used to visualize the distribution and variability of **Billing Amounts, Patient Age, and Room Numbers**, supporting comparative analysis.

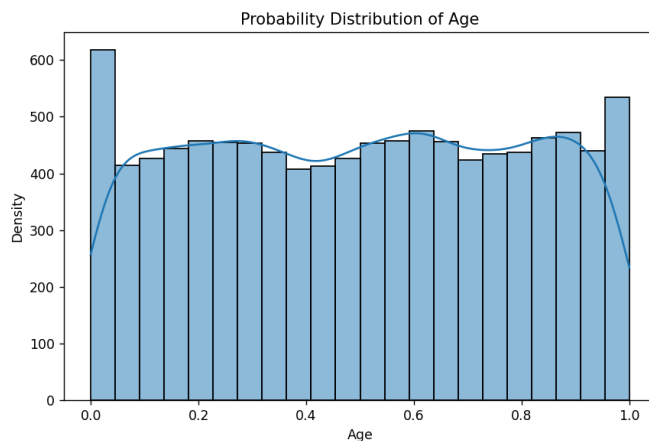
Overall Insight:

The interactive dashboard provides an intuitive platform to explore healthcare trends, identify cost variations, and understand patient demographics effectively.



8.2 Probability Analysis

Range and variance were calculated for key numerical variables such as **Age**, **Billing Amount**, and **Room Number** to understand their spread and variability. Histograms with KDE curves were plotted to analyze the probability distribution of these variables, helping to identify data concentration, dispersion, and overall distribution patterns within the healthcare dataset.



Conclusion

This project successfully conducted an end-to-end Exploratory Data Analysis on the healthcare dataset. The data was cleaned, transformed, and analyzed to uncover meaningful patterns related to patient demographics, hospital admissions, and billing behavior. Visualizations and

descriptive statistics provided clear insights into healthcare utilization trends. Overall, this analysis demonstrates how EDA can support better decision-making in healthcare management and improve operational efficiency.

File Structure:

